# Threshold Settlement: Escrow and Payout Without a Key

Zach Kelling

*Hanzo AI*

2026

**Abstract**

Every compute market eventually holds someone else's money. A job is posted, funds are escrowed, work is performed, a verifier decides, and a payout is issued. The question nobody enjoys asking is who holds the key that releases the escrow.

The answer should be nobody. We describe a settlement architecture in which the private key authorising payouts never exists — not in memory, not on disk, not during signing — and in which the operator set can change without the key ever being reconstructed. We give the protocol table we run in production across twenty or more chains, including two post-quantum lattice families and a composition requiring both, and report measured signing and key-generation latencies.

We then describe the interface boundary that makes this composable with a market rather than entangled with it: an assembler that turns a payment specification into an unsigned transaction and a set of digests, and a finaliser that turns digests plus signatures back into a broadcastable transaction. The market decides what to pay. The threshold cluster decides whether to sign. Neither needs to understand the other.

# Contents

# 1 The custody question

Escrow in a decentralized market is usually solved one of three ways, and two of them are bad.

A single operator key is simple and is a single point of theft, a single point of coercion and a single point of loss. A smart contract on the settlement chain is genuinely good where the asset is native to that chain, and useless where it is not — which is the common case as soon as providers want paying in something that lives elsewhere. A multi-signature scheme is better than one key but publishes the entire policy on chain, costs linearly in signers, and is unavailable or awkward on many chains.

Threshold signing gives the properties of multi-signature without the disclosure, the cost or the chain dependence: $t$ of $n$ parties cooperate to produce a signature that is indistinguishable from a single-key signature, and the full key is never assembled at any point in the protocol.

**Definition 1** (No-reconstruction property)**.** *A threshold signing scheme has the no-reconstruction property if there exists no point in key generation, signing, or resharing at which any single party holds material sufficient to recover the private key, and no protocol transcript from which fewer than $t$ colluding parties can recover it.*

This is stronger than secret sharing with reassembly at signing time, which is what several deployed systems actually do and which reintroduces the single point of theft for the duration of every signature.

# 2 The protocol table

| Protocol | Curve or lattice | Settlement reach |
|---|---|---|
| CGGMP21 | secp256k1 | Bitcoin, Ethereum, all EVM chains, XRPL |
| FROST | Ed25519 | Solana, TON, Polkadot, Cardano, Substrate |
| BLS | BLS12-381 | aggregate attestation, beacon signatures |
| SR25519 | Ristretto255 | Substrate native |
| Pulsar | M-LWE (threshold ML-DSA-65) | post-quantum, anchors and custody |
| Corona | R-LWE | post-quantum, orthogonal lattice family |
| Double lattice | Pulsar $\oplus$ Corona | both must verify |

Table 1: Threshold protocols in production, with the settlement surfaces each reaches.

Three properties of this table matter for a market.

**Non-interactive signing.** CGGMP21 requires an interactive key generation once, after which signing is non-interactive. For a market issuing many payouts this is the difference between a settlement path that scales and one that turns into a coordination problem at load. It also produces signatures bit-identical to any standard secp256k1 signature, so nothing downstream needs to know a threshold scheme was involved.

**Dynamic resharing.** Parties can be added or removed without downtime and without reconstructing the key. In a permissionless market the operator set churns; a custody design that requires a ceremony every time it changes will not survive contact with reality.

**Post-quantum options with a hedge.** A custody key is the longest-lived, highest-value secret in the system, which makes it the worst possible thing to leave on an elliptic curve. Classical threshold ECDSA is $t$-of-$n$ against a classical adversary and 0-of-$n$ against Shor. The lattice families provide a post-quantum threshold; the double-lattice composition, which concatenates a

Pulsar and a Corona signature and requires both to verify, hedges against a break in one lattice problem rather than in lattices generally.

## 3 Measured behaviour

Signing completes in under 25 ms and key generation in 12 ms to 82 ms, with Byzantine fault tolerance up to $t-1$ malicious parties. For a settlement path these are not the binding constraint — block times dominate by two to three orders of magnitude — which is the comfortable position to be in.

## 4 The interface boundary

The design decision that makes this composable is refusing to let the signer know what a job is.

```
PreSign(spec) -> (unsigned transaction, per-input digests)
Finalize(unsigned, sigs) -> (raw transaction, txid)
```

The assembler takes a payment specification — source, destination, amount, fee policy — and returns a wire-correct unsigned transaction together with the digests that must be signed, computed under the correct sighash rules for the chain. The finaliser takes those digests' signatures and produces a broadcastable transaction. Between the two calls sits whatever authorisation process the operator wants, and the assembler neither knows nor cares what it is.

This yields a clean separation for a market:

- The *allocator* decides who should be paid and how much. It has no access to key material and cannot move funds.
- The *verifier* decides whether the work was done. It produces an attestation, not a signature.
- The *threshold cluster* decides whether to sign a specific transaction against a specific attestation policy. It has no opinion about pricing.

Each can be audited, replaced, or run by a different party. A compromise of the allocator produces mispriced jobs, not stolen funds. A compromise of the verifier produces incorrect attestations, which the signing policy can require multiple of.

## 5 Per-job escrow

We recommend deriving a distinct escrow address per job rather than pooling.

$$\text{escrow}(J) \;=\; \text{derive}\big(\text{master},\ \text{path}(J)\big), \tag{1}$$

with a deterministic path incorporating the job identifier. The properties this buys are worth the small extra bookkeeping. Funds for distinct jobs are isolated, so a policy failure on one job cannot drain another. The escrow address is computable by the buyer before funding, so the buyer can verify where its money is going without trusting an API response. Settlement is a single threshold signature over a transaction whose specification the verifier has already validated. And an unclaimed escrow is visibly unclaimed on chain, which makes refund logic auditable rather than a database state.

The derivation should use a hardened path structure with one seed and many paths rather than many seeds. Maintaining $n$ independent seeds multiplies custody risk with no compensating benefit; independent authorisation boundaries are better enforced at the access-control layer, where they can be scoped and revoked, than by seed proliferation.

## 6 What a market has to decide

Threshold signing removes the key-holder question but does not answer the policy question, and the policy is where the real design work is.

**What does the cluster require before signing?** A verifier attestation is the minimum. Two independent attestations, or an attestation plus an elapsed challenge window, are stronger. The window matters: it converts an undetected verifier compromise into a race the honest party can win.

**What is the refund path?** A job whose deadline passes without a valid result must return funds. This should be a signing policy the cluster can execute against an on-chain timestamp, not a manual operation.

**What is the dispute path?** If the buyer and verifier disagree, whose attestation governs? We suggest the answer is neither: escalate to a higher verification tier and let the result decide, with the loser paying for it. That makes frivolous disputes self-limiting.

**Who runs the parties?** The security of a $t$-of-$n$ scheme is the security of the least correlated $t$ parties. Parties operated by one entity, on one cloud, in one region, under one jurisdiction are not $n$ parties. This is the same diversity problem replication has, and it deserves the same explicit treatment rather than being assumed away.

## 7 Availability

The threshold signing engine, the multi-chain adapters, the transaction assembler and the key-management integration are open source under permissive licence. The signing engine can be adopted without the assembler, and the assembler without the market — the boundary described above is real, not aspirational, and each piece is separately useful.